DOCUMENTO DE PROJETO DO SISTEMA

SYSTEM DESIGN DOCUMENT

Projeto	TaskAnalyzer
Aluna	Kamila Gomes da Silva
Curso	Ciência da Computação
Instituição / Turma	CEUB — Asa Norte | Turma A
Versão do documento	1.0
Data	24/10/2026
Status	Proposta de arquitetura — validar contra a implementação

1. Introdução

Este documento descreve a proposta de arquitetura e o desenho técnico do TaskAnalyzer, um sistema que recebe tarefas e gera métricas de produtividade. O documento organiza os componentes, o modelo de dados, o fluxo de processamento, as validações e a estratégia de testes de forma rastreável à especificação do projeto.

2. Objetivo e escopo

O objetivo do TaskAnalyzer é analisar uma coleção de tarefas e gerar indicadores de produtividade, incluindo tempo médio de conclusão, taxa de atraso, contagens e indicadores agrupados por prioridade e indicadores gerais.

O sistema deve respeitar as restrições do projeto: Python 3.11 ou superior, tipagem explícita em funções e métodos, responsabilidade única, conformidade com PEP 8, docstrings no padrão Google, exceções específicas e logging estruturado. Não deve introduzir persistência em arquivos ou banco de dados nem funcionalidades não autorizadas pela especificação.

3. Premissas e limitações

A arquitetura assume processamento em memória dos dados fornecidos ao sistema. Não foi especificada uma interface gráfica, API web, banco de dados ou mecanismo de armazenamento permanente; portanto, nenhum desses elementos é incluído como requisito.

A especificação define que uma tarefa sem data de conclusão é considerada pendente. Também exige tratamento de datas inválidas, prevenção de divisão por zero e arredondamento numérico para duas casas decimais.

Pendência importante — taxa de atraso: os campos conhecidos da tarefa não incluem uma data de vencimento ou um prazo explícito. A definição de quando uma tarefa está atrasada precisa ser esclarecida com o responsável pela especificação. Até essa confirmação, o cálculo da taxa de atraso não deve receber uma regra inventada silenciosamente.

4. Requisitos arquiteturais

Identificador	Requisito / decisão	Implicação no desenho
RA-01	Python 3.11 ou superior	Executar e testar em ambiente compatível com Python 3.11+.
RA-02	Type hints em todas as funções e métodos	Declarar os tipos dos parâmetros e retornos.
RA-03	Responsabilidade única	Separar validação, cálculos e coordenação do processamento quando isso corresponder ao código.
RA-04	PEP 8 e docstrings Google	Manter estilo consistente e documentar funções e métodos.
RA-05	Exceções específicas e logging	Reportar falhas de validação de modo explícito e registrar eventos úteis.
RA-06	Sem persistência	Não armazenar dados em arquivos nem banco de dados.

5. Arquitetura lógica proposta

A arquitetura lógica divide o processamento em responsabilidades. Os nomes abaixo são papéis conceituais; os nomes de módulos, classes e funções definitivos devem refletir a implementação real.

Componente lógico	Responsabilidade
Entrada / coordenação	Receber a coleção de tarefas e coordenar o processamento.
Validação	Verificar presença, tipos e validade dos campos conforme a especificação.
Análise e cálculos	Calcular tempo médio, contagens por prioridade e indicadores gerais usando dados válidos.
Tratamento de erros	Lançar ou propagar exceções específicas para entradas inválidas e registrar eventos pertinentes.
Resultado	Devolver os indicadores calculados ao chamador, sem persistência permanente.

Fluxo lógico: entrada das tarefas → validação → cálculo das métricas → consolidação dos indicadores → retorno dos resultados. Se houver dados inválidos, o fluxo deve interromper ou rejeitar o processamento conforme a regra de erro definida pela especificação.

6. Modelo de dados

Campo	Significado	Observações
id	Identificador da tarefa	Deve respeitar o tipo e as regras definidos na especificação original.
titulo	Título descritivo	Campo textual conforme validação prevista.
prioridade	Prioridade da tarefa	Valores válidos devem seguir a especificação; não presumir uma enumeração sem confirmação.
data_criacao	Data/hora de criação	Usada como referência temporal para o cálculo de conclusão.
data_conclusao	Data/hora de conclusão ou nulo	Ausência indica tarefa pendente.
status	Estado da tarefa	Valores válidos e relação com data_conclusao devem seguir a especificação.

A estrutura acima descreve os campos informados no projeto. Não acrescenta campos como prazo, vencimento ou responsável, pois não fazem parte do modelo confirmado até o momento.

7. Regras de negócio e cálculos

Tempo médio de conclusão. Para tarefas concluídas e com datas válidas, o tempo de conclusão corresponde à diferença entre data_conclusao e data_criacao. O tempo médio é calculado sobre o conjunto de tarefas concluídas elegíveis. A unidade de medida e o comportamento quando nenhuma tarefa pode ser incluída devem seguir a especificação e os testes.

Taxa de atraso. A métrica é solicitada, mas o critério que classifica uma tarefa como atrasada não está suficientemente definido nos campos conhecidos. É necessária uma decisão formal antes de fixar fórmula e testes.

Indicadores por prioridade. Agrupar as tarefas por prioridade e produzir as contagens/indicadores solicitados. As categorias e métricas exatas devem corresponder à especificação original.

Indicadores gerais. Consolidar os indicadores definidos para o conjunto de tarefas analisadas.

Precisão e casos-limite. Resultados numéricos devem ser arredondados para duas casas decimais quando previsto; o sistema deve evitar divisão por zero e tratar datas inválidas com exceções específicas.

8. Validação e tratamento de erros

A validação deve ocorrer antes de utilizar os dados nos cálculos. A implementação deve verificar os tipos e campos exigidos, validar datas e respeitar as regras sobre tarefas pendentes. Para cada condição de erro, devem ser utilizadas as exceções e mensagens previstas na especificação original.

O logging deve registrar informações técnicas úteis para diagnóstico, sem incluir dados pessoais desnecessários. A forma exata das mensagens e das exceções deve ser confirmada com a especificação e com os testes automatizados.

9. Estratégia de testes

Grupo	Objetivo	Resultado esperado
Cálculo válido	Verificar métricas com tarefas válidas e concluídas.	Valores correspondentes aos cálculos de referência, com precisão prevista.
Tarefas pendentes	Verificar tarefas sem data de conclusão.	Serem consideradas pendentes e não tratadas como concluídas.
Datas inválidas	Fornecer datas inválidas ou inconsistentes.	Exceção específica conforme a especificação.
Divisão por zero	Executar cenários sem elementos elegíveis para uma taxa.	Não ocorrer erro matemático não tratado; comportamento conforme especificação.
Prioridade e visão geral	Verificar agrupamentos e consolidação.	Contagens e indicadores coerentes com os dados de entrada.

Os testes devem ser executados localmente e, se configurado, pelo GitHub Actions. O documento não afirma que os testes passaram; o estado deve ser comprovado pela execução real e pelas evidências anexadas à entrega.

10. Integração contínua e execução em contêiner

GitHub Actions. A automação proposta deve ser acionada em push e pull request, configurar Python 3.11, instalar as dependências declaradas e executar os testes automatizados. A configuração definitiva deve corresponder aos arquivos do repositório.

Docker. O Dockerfile e o docker-compose.yml podem encapsular o ambiente de execução e executar a suíte de testes de modo reproduzível. Docker não substitui os testes; ele padroniza o ambiente em que são executados.

Evidências a anexar após execução: saída real dos testes locais, execução verde/vermelha do workflow no GitHub Actions, saída do comando Docker e identificação do commit analisado.

11. Segurança, ética e revisão humana

O sistema deve processar apenas os dados necessários para produzir os indicadores. Logs devem evitar conteúdo sensível e informações pessoais desnecessárias. As métricas precisam ser explicáveis e verificáveis; indicadores de produtividade não devem ser interpretados automaticamente como avaliação completa do desempenho individual.

Se ferramentas de inteligência artificial forem utilizadas para apoiar a implementação, suas sugestões devem ser revisadas por uma pessoa, comparadas à especificação e validadas por testes. Código gerado por IA não é evidência de conformidade por si só.

12. Rastreabilidade entre requisitos e componentes

Requisito	Componente responsável	Validação
Entrada e validação das tarefas	Entrada / validação	Testes de campos, tipos e datas.
Tempo médio de conclusão	Análise e cálculos	Testes com resultados esperados e casos sem tarefas elegíveis.
Taxa de atraso	Análise e cálculos	Bloqueada até a regra de atraso ser definida; depois, testar fórmula e limites.
Indicadores por prioridade e gerais	Consolidação dos indicadores	Testes de agrupamento e contagem.
Exceções e logging	Validação / tratamento de erros	Testes de falhas e inspeção dos registros.

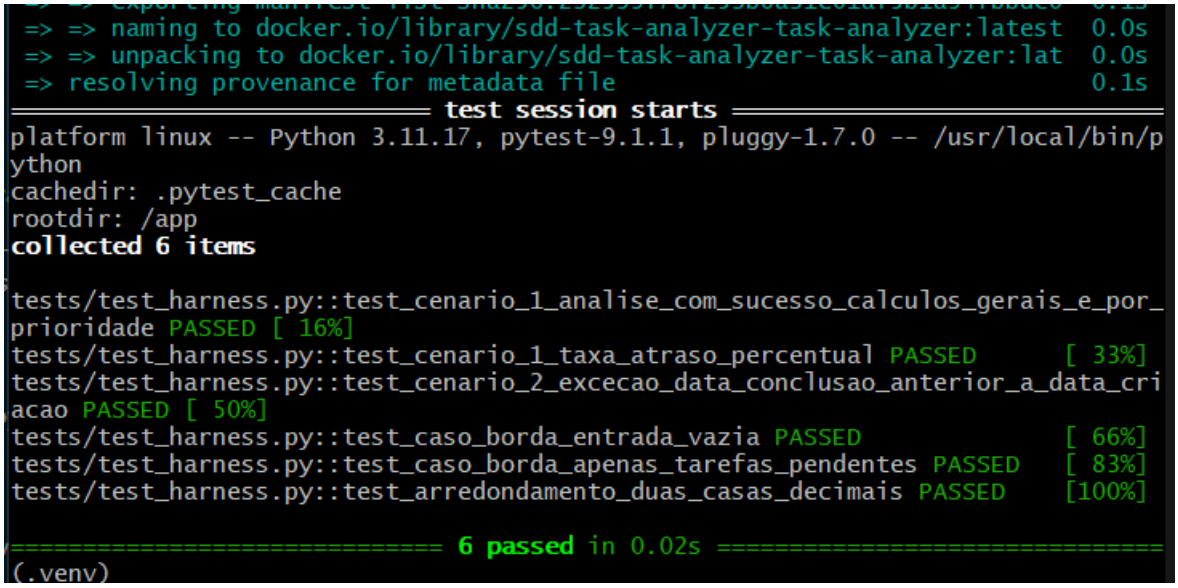
=

13. Conclusão

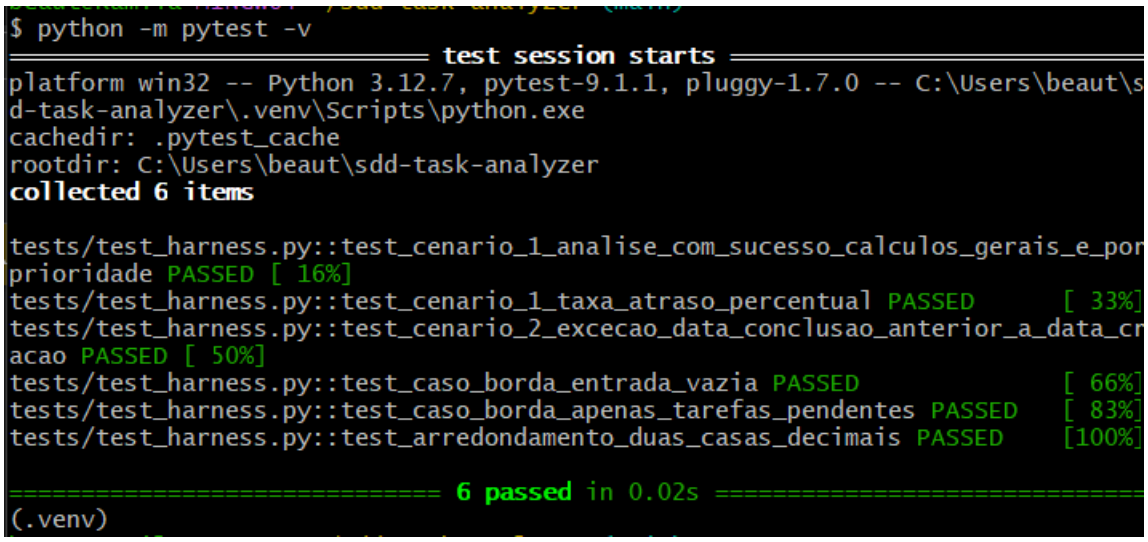
O desenho proposto organiza o TaskAnalyzer em responsabilidades claras, prioriza validação de entradas, cálculo verificável de métricas, tratamento explícito de erros e testes automatizados.

14. Comprovação dos testes

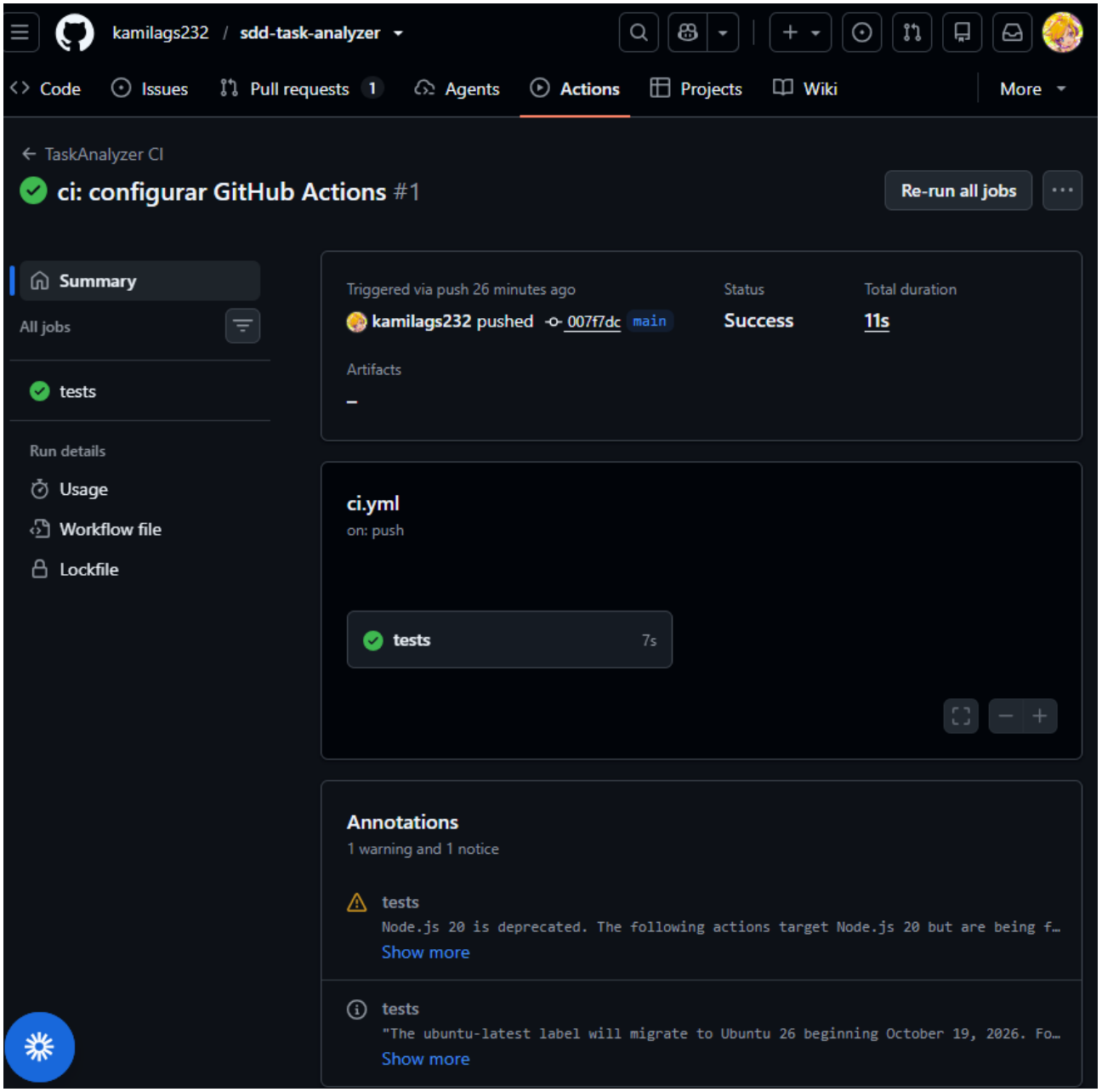
docker compose run --build --rm task-analyzer



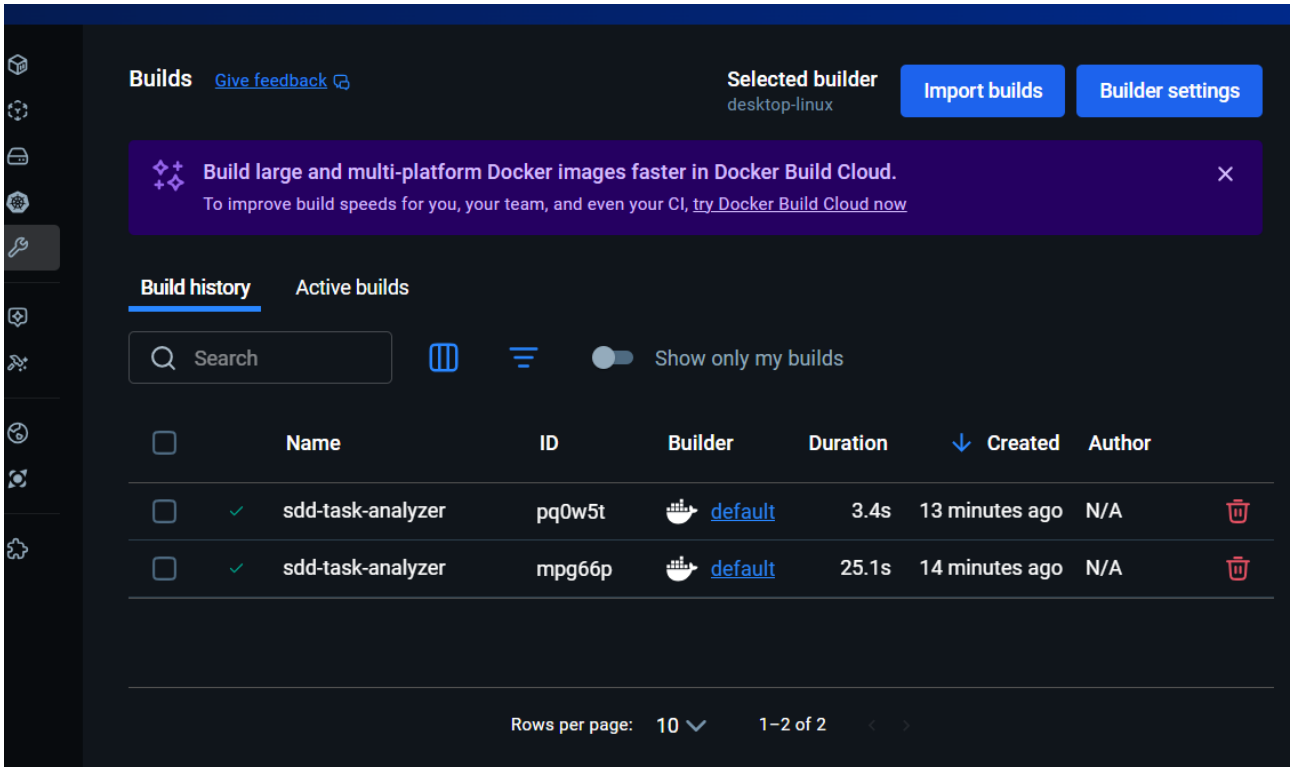
pytest



Github Actions


kamilags232 / sdd-task-analyzer
Code
Issues
Pull requests 1
Agents
Actions
Projects
Wiki
More
TaskAnalyzer CI
ci: configurar GitHub Actions #1
Re-run all jobs
Summary
All jobs
tests
Run details
Usage
Workflow file
Lockfile
Triggered via push 26 minutes ago
kamilags232 pushed 007f7dc main
Status
Success
Total duration
11s
Artifacts
–
ci.yml
on: push
tests 7s
Annotations
1 warning and 1 notice
tests
Node.js 20 is deprecated. The following actions target Node.js 20 but are being f…
Show more
tests
"The ubuntu-latest label will migrate to Ubuntu 26 beginning October 19, 2026. Fo…
Show more

Docker


Builds
Give feedback
Selected builder
desktop-linux
Import builds
Builder settings
Build large and multi-platform Docker images faster in Docker Build Cloud.
To improve build speeds for you, your team, and even your CI, try Docker Build Cloud now
Build history
Active builds
Search
Show only my builds
Name
ID
Builder
Duration
Created
Author
sdd-task-analyzer
pq0w5t
default
3.4s
13 minutes ago
N/A
sdd-task-analyzer
mpg66p
default
25.1s
14 minutes ago
N/A
Rows per page:
10
1–2 of 2